Database Development and Design (SOE201)

Lecture 1b: Data Storage Structures

Dr. Wei Wang

AIAT

Learning Outcomes

- Overview of Physical Storage Media
- Magnetic Disks
- SSD (Solid-State Drive)
- Storage Access
- Record Organisation
- File Organisation

Building Database: High-Level

- Design conceptual schema using data model.
  - Design relational schema, e.g. student (sid, name)
- Data Definition Language (DDL)
  - *CREATE TABLE student (sid char(8) primary key, name varchar(32))*
- Data Manipulation Language (DML)
  - *INSERT INTO student VALUES ('00112233', 'Paul')*

Where and How is All Data Stored?

- Actual data (records), also known as tuples
- Metadata: tables, attributes, data types, constraints…
- Transaction logs
- Indices
- Statistical data
- etc.

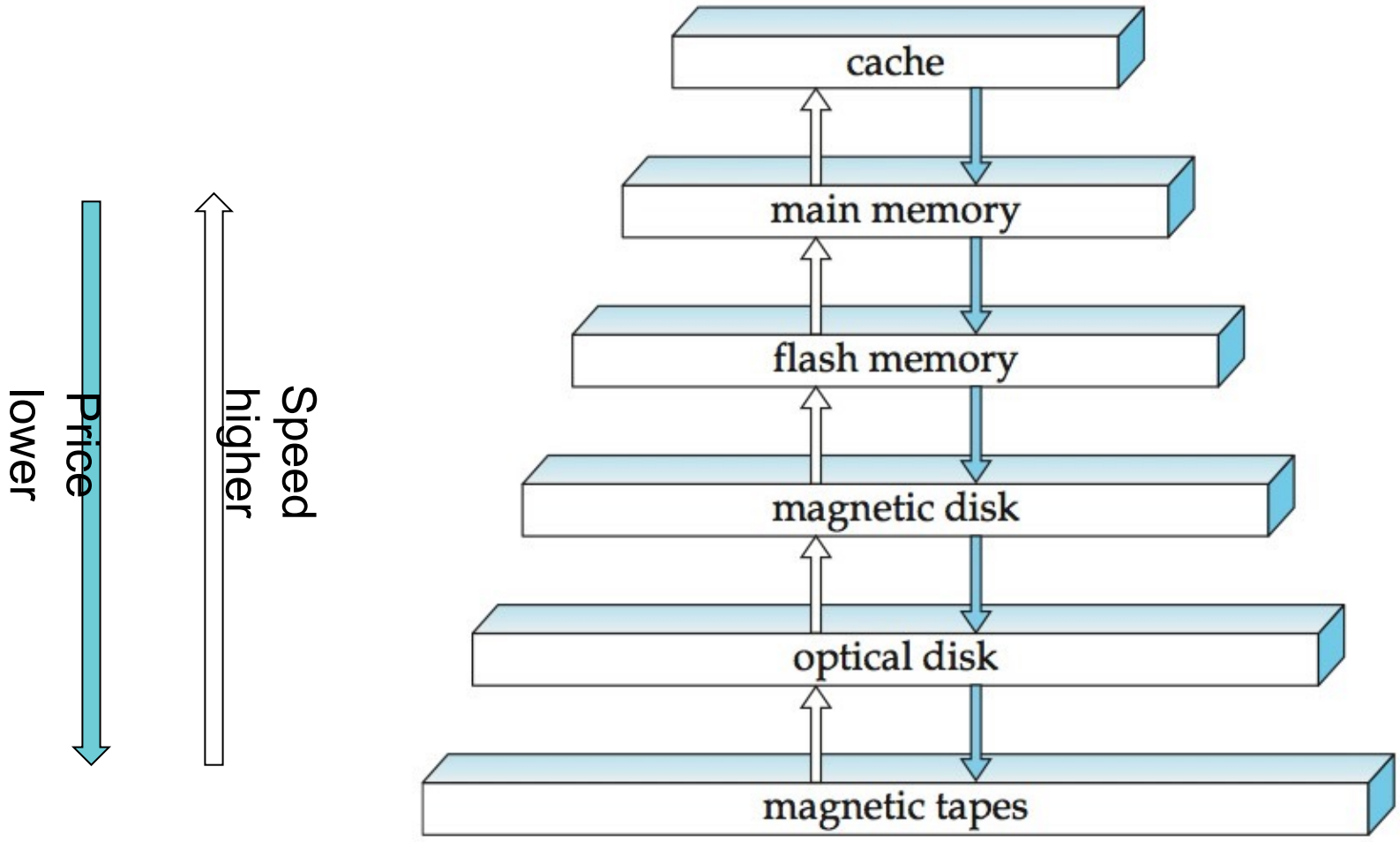
All of them need to be stored on some sorts of physical storage media.

Classification of Physical Storage Media

- Classification:
  - Volatile storage
    - Loses contents when power is switched off, e.g. memory
  - Non-volatile storage
    - Contents persist even when power is off, e.g. secondary and tertiary storage
- Factors affecting choice of storage media include
  - Speed with which data can be accessed
  - Cost per unit of data
  - Reliability

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Physical Storage Media



Xi'an Jiaotong-Liverpool University 西交利物浦大学

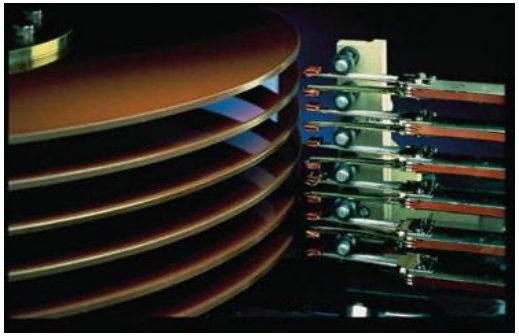
Physical Storage Media Hierarchy

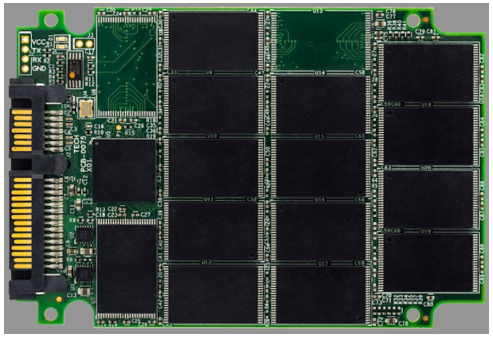
- Primary storage: fastest media but volatile (cache, main memory)
- Secondary storage: next levels in hierarchy, non-volatile, moderately fast access time
  - Also called on-line storage
  - e.g. flash memory, magnetic disks
- Tertiary storage: lowest levels in hierarchy, non-volatile, slow access time
  - Also called off-line storage and used for archival storage
  - e.g. magnetic tape, optical storage

Main Memory, Magnetic Disks and SSD

Physical View

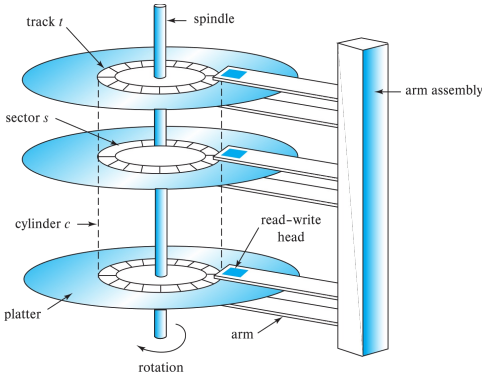


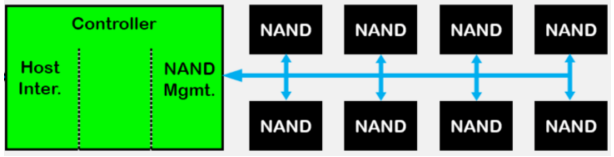




Logical View



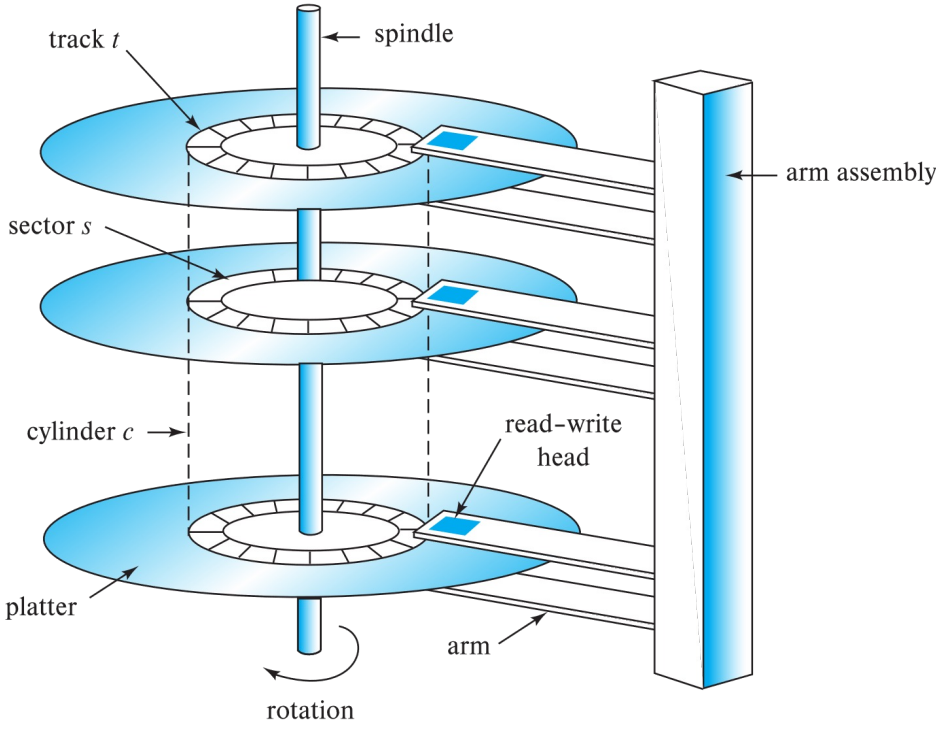




SSD: Solid State Drive
NAND: NAND flash memory

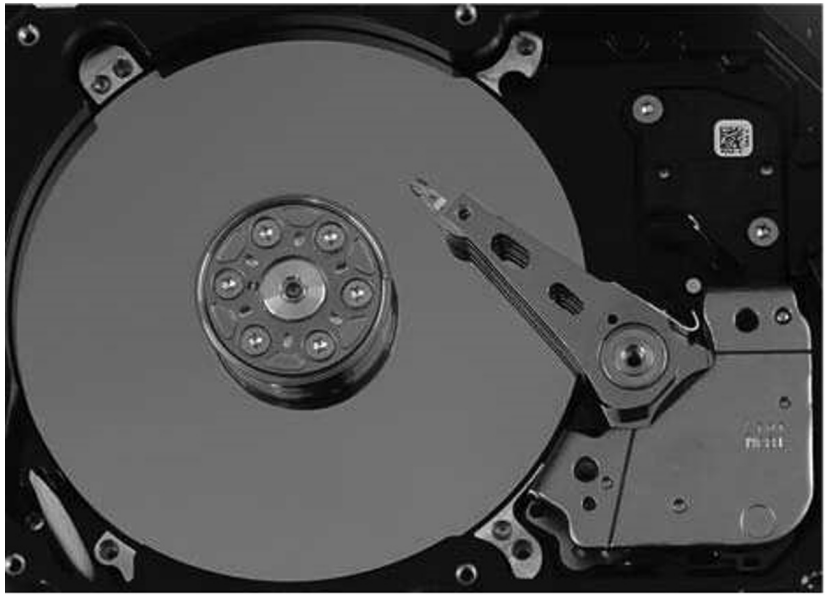
Magnetic Disks

- Read-write head
  - Positioned very close to platter surface (almost touching it)
  - Reads or writes magnetically encoded information.
- Surface of platter divided into circular tracks
- Each track is divided into sectors.
- Sector is smallest unit of data that can be read or written.
- Cylinder *i* consists of *i*th track of all platters



Disk Drive Read/Write Operation

- Disk arm swings to position head on right track;
- Platter spins continually to find right sector;
- Data is then read/written as sector passes under head.
  - Read/Write needs time!



Swing for track and spin for sector

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Access Time

- Access time is time taking from when read or write block request is issued to when data transfer begins (or ends).
- Access time = Seek time + Rotational latency + (Transfer time)
- Seek time – time taken to position arm over right track.
  - Average seek time is about 1/2 of worst case seek time (e.g. from innermost to outermost track)
- Rotational latency – time taken for sectors to be accessed to appear under head.
  - Average latency is about 1/2 of worst case latency (e.g. nearly 360 degree rotation)
- Transfer time – time to actually read/write data in sectors.
  - *Usually estimated by time for disk to rotate over sectors.*
  - In most cases, transfer time is much less than seek time and rotational latency.

Disk Block

- Block - *contiguous* sequence of sectors from single track
- Data is transferred between disk and main memory in blocks
- Typical sizes range from 4 to 16 kilobytes
  - Smaller blocks: more transfers from disk
  - Larger blocks: more space wasted due to partially filled blocks

Optimisation of Disk-Block Access

- Disk-arm scheduling orders pending accesses to tracks so that disk arm movement is minimised.
- Elevator algorithm:
  - Moves disk arm in one direction (from outer to inner tracks or vice versa),
  - Processes next request in that direction, till no more requests in that direction, then
  - Reverses direction and repeat

Flash Memory and SSD

- Two types of flash memory: NOR and NAND
- NAND flash predominantly used for data storage.
  - Reading from NAND requires entire page, commonly 4,096 bytes;
  - Pages in NAND flash similar to sectors in magnetic disk.
- Solid-State Drives (SSDs) are built using NAND flash and provide same block-oriented interface as disk storage.

SSD

- Much faster random access, but more expensive compares to HDDs
  - Used as primary boot drive in personal computers nowadays.
  - Latency to retrieve a page ranges from 20 to 100 microseconds
  - Data transfer rate ranges from
    - Around 500 megabytes per second with SATA interfaces
    - Up to 3 gigabytes per second using NVMe PCIe interfaces
- Write to page of flash memory typically takes about 100 microseconds.
  - However, page has to be erased first and rewritten subsequently.

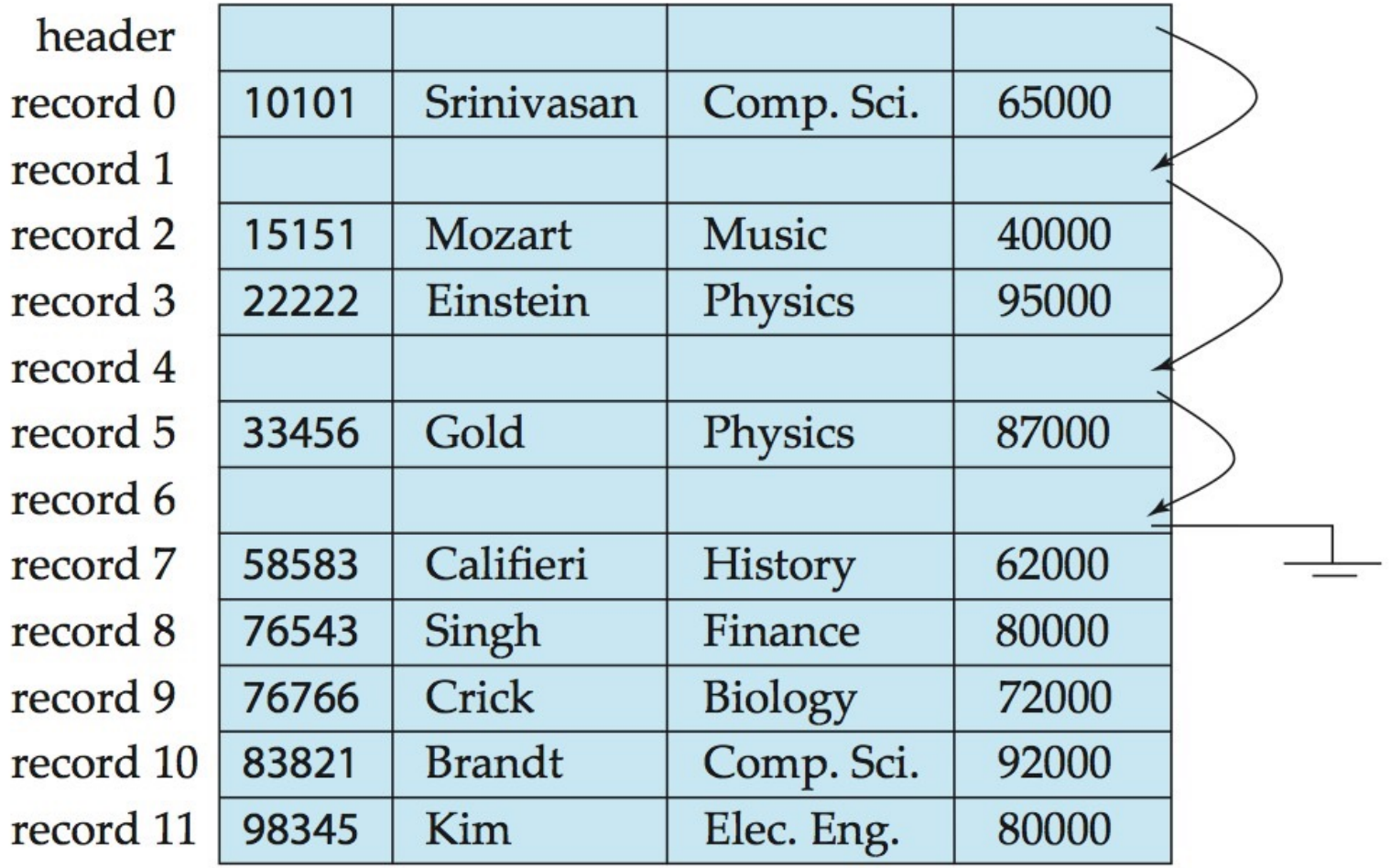
RAID

- RAID (Redundant Arrays of Independent Disk)
  - Have large number of disks in system
  - Independent reads or writes can be performed in parallel
  - Achieve improved performance
    - Improvement of Reliability via Redundancy
      - Store extra information that is not needed normally but can be used in event of failure to rebuild lost information, e.g. mirroring or shadowing
    - Improvement in Performance via Parallelism
      - Improve transfer rate by striping data across multiple disks.

File Organisation

- Logically, database is stored as collection of files; each file is sequence of records; record is sequence of fields.
- Physically, each file is partitioned into blocks (pages).
  - File organisation: optimise block access time by organising blocks to correspond to how data will be accessed.
- ***Assumption***: "*no record is larger than a block, i.e. each record is entirely contained in a single block.*"
- Two ways for storing records
  - Fixed-length records
  - Variable-length records

Fixed-length Records



header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-length Records - cont'd

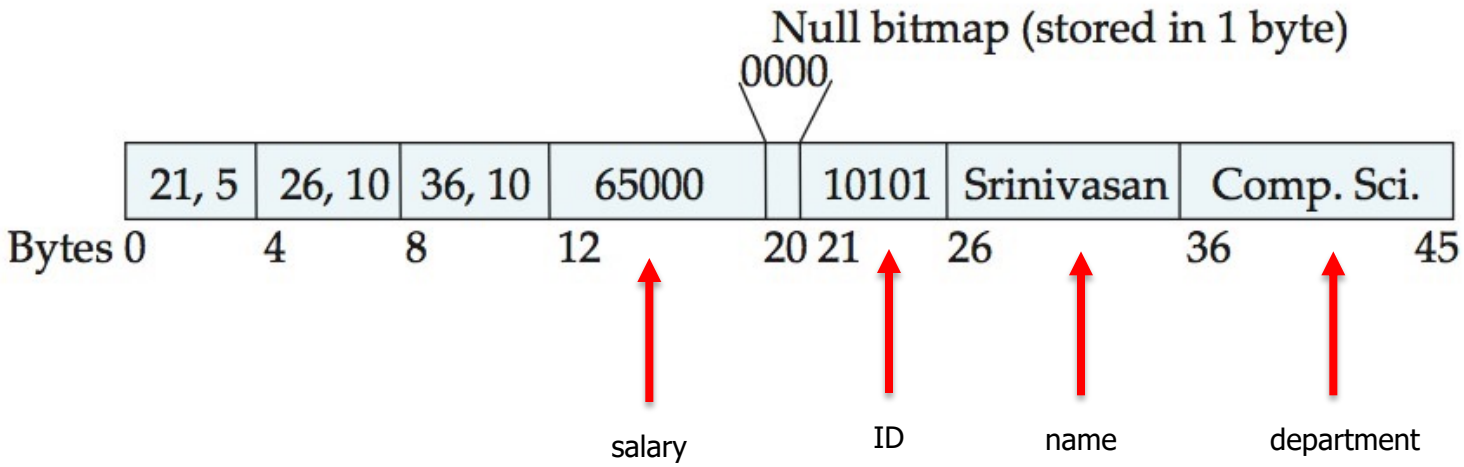
- Record size is fixed; allocate only as many records as would fit entirely in block
- Undesirable to delete records (requires additional block accesses, i.e. moving data to fill up space)
  - Solutions
    - Allocate certain number of bytes as file header.
    - Deleted records form linked list, often referred to as free list.
    - On insertion of new record, use record pointed to by header.
      - Change header pointer to point to next available record.
      - If no space is available, add new record to end of file.

Variable-length Records

- Representation of record with variable-length attributes has two parts:
  - Initial part with fixed length information
  - Data for variable-length attributes
- Variable-length attributes are represented in initial part by pair (offset, length)
  - Offset denotes where data for that attribute begins within record
  - Length of variable-sized attribute in bytes.

Variable-length Records - cont'd

- Instructor record:
  - First three attributes ID, name, and department are variable-length strings
  - Fourth attribute salary is fixed-sized number

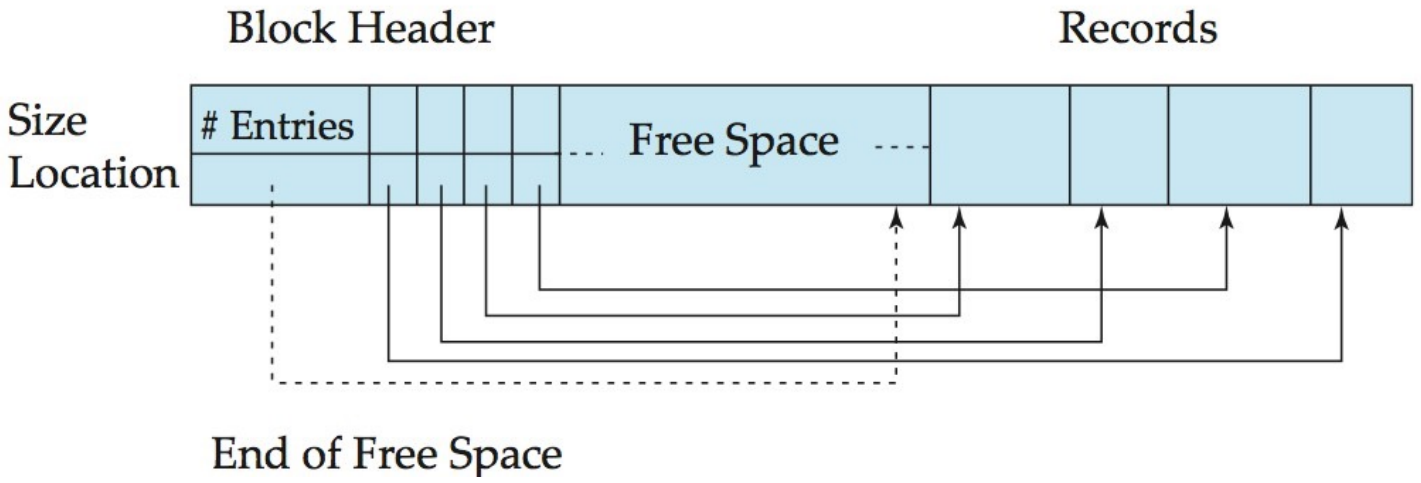


Variable-length Records - cont'd

- Storing variable-length records in block uses slotted-page structure:
  - Header
    - Number of record entries.
    - End of free space pointer.
    - Array whose entries contain location and size of each record.
  - Actual records are allocated contiguously in block, starting from end of block.
  - Free space is always contiguous, between final entry in header array, and first record.

Slotted-page Structure

- If record is inserted, space is allocated for it at end of free space.
- If record is deleted, space that it occupies is freed.
  - Further, records in block before deleted record are moved, so that all free space is again between final entry in header array and first record.



Organisation of Records in Files

- Heap – record can be placed anywhere in file where there is space.
- Sequential – store records in sequential order, based on value of search key of each record.
- Multitable clustering - records of several different relations can be stored in same file.
- B+-tree - provide efficient ordered access to records even with large number of insert, delete, or update operations (more in future).
- Hashing – hash function computed on search key; result specifies in which block of file record should be placed (more in future).

Heap File Organisation

- Records can be placed anywhere in file where there is free space; usually do not move once allocated.
- Need to efficiently find free space within file
- Free-space map
  - Array with 1 entry per block. Each entry is few bits to byte, and stores fraction of block that is free.
    - In example below, 3 bits per block, value divided by 8 indicates fraction of block that is free.

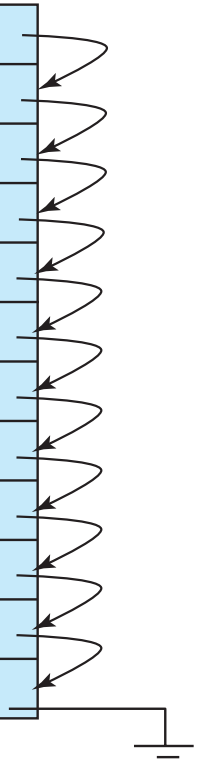
4	2	1	4	7	3	6	5	1	2	0	1	1	0	5	6

    - Can have second-level free-space map.
  - Free space map written to disk periodically
    - OK to have wrong (old) values for some entries
    - Will be detected and fixed later

Sequential File Organisation

- Suitable for applications that require sequential processing of entire file.
- Records are ordered by search-key (ID in following example).

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



Sequential File Organisation – cont'd

- Deletion – use pointer chains
- Insertion – locate position where record is to be inserted
  - If there is free space, then insert there
  - If no free space, insert record in overflow block
  - In either case, pointer chain must be updated
- Need to re-organise file from time to time to restore sequential order

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Multitable Clustering File Organisation

- Store several different relations in one file
  - department

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Physics	Watson	70000

  - instructor

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

  - multitable clustering of *department* and *instructor*

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

Multitable Clustering File Organisation cont'd

- Good for queries involving join (e.g. join of *department* and *instructor*)
- Bad for queries involving only one relation
- Results in variable size records
- Can add pointer chains to link records of particular relation

End of Lecture

- Reading
  - Chapters 12 and 13, database system concepts, 7th ed.